

Design and Development of an Underwater Event Detection System Based on Embedded AI

Location: ENIB, Lab-STICC, Brest

AKSHARA SOMAN

18.09.2025

Internship report written and submitted in fulfillment of the requirements
for the degree of Master of Science
MSc. Automotive Embedded Systems

Under the Guidance of:

R. KHEMMAR
MSc. AES Responsible
ESIGELEC

A. BENZINOU
K. NASREDDINE
H.N. TRAN
Lab-STICC Responsibles



Acknowledgment

I would like to express my deepest gratitude to all those who have contributed to the successful completion of my internship and this thesis.

First and foremost, I am sincerely thankful to my direct supervisors at Lab-STICC, UMR CNRS 6285, hosted at ENIB — **Mr. Abdesslam Benzinou**, **Mr. Kamal Nasreddine**, and **Mr. Hai Nam Tran** — for their continuous support, guidance, and encouragement throughout my internship. Their valuable insights, expertise, and constructive feedback have been instrumental in enhancing my technical skills and broadening my research perspective. This work is supported by EUR ISblue (Interdisciplinary graduate School for the Blue Planet).

I would also like to extend my heartfelt gratitude to **Mr. Redouane Khemmar**, MSc AES responsible at ESIGELEC, for his academic guidance and encouragement. His supervision and mentorship have been a source of constant motivation and played a significant role in shaping both the direction and the outcomes of this work.

Furthermore, I am grateful to the entire Lab-STICC team at ENIB for their warm welcome, collaborative spirit, and the stimulating environment they provided, which made this journey both enriching and enjoyable.

On a personal note, I would like to express my profound gratitude to my **family and friends** for their unwavering support, encouragement, and belief in me. Their constant motivation helped me overcome challenges and remain focused throughout this journey.

This internship and thesis would not have been possible without the guidance, support, and encouragement of all the individuals mentioned above. I am truly thankful and hope to honor their efforts by applying the knowledge and skills I have gained in my future endeavors.

Thank you all for your unwavering support.
AKSHARA SOMAN

Contents

1	CONTEXT AND OBJECTIVES	1
1.1	Context	1
1.2	Objectives	1
2	ENTERPRISE PRESENTATION	2
2.1	Overview of the Lab-STICC Research Laboratory	2
2.2	Laboratory Overview	2
2.2.1	Key Details	2
2.2.2	Historical Background	3
2.3	Presentation of the Laboratory	3
2.3.1	Regional Distribution	3
2.3.2	Core Operations and Stakeholders	4
2.3.3	Organizational Framework	4
2.4	Presentation of the Host Department	6
2.4.1	AI & Oceans Division	6
3	MISSION WITH ENTERPRISE	8
3.1	Introduction to the Mission	8
3.1.1	Challenges in Underwater AI Event Detection	8
3.1.2	Existing Approaches and Related Work	9
3.1.3	Mission Reframing	10
3.2	Project and Industry Context	10
3.2.1	Industry Relevance of AI in Marine Applications	10
3.2.2	Current Technological Gaps	11
3.2.3	Positioning of This Project	12
3.3	My Role and Responsibilities	12
3.3.1	Literature Review and Benchmarking	12
3.3.2	Dataset Preparation and Preprocessing	13
3.3.3	Algorithm Development and Training	13
3.3.4	Model Optimization for Raspberry Pi 5	14
3.3.5	Evaluation and Result Analysis	14
3.3.6	Collaboration and Reporting	14
3.3.7	Summary	15
3.4	Dataset Preparation and Preprocessing	15
3.4.1	Importance of Dataset Preparation in Underwater AI	15
3.4.2	Dataset Labeling	16
3.4.3	Dataset Cleaning	16
3.4.4	Input Resolution Variations	16
3.4.5	Normalization and Standardization	17
3.4.6	Challenges in Dataset Preparation	17
3.5	Algorithm Development and Training	18

3.5.1	Rationale for Choosing YOLOv11	18
3.5.2	Model Variants Explored	18
3.5.3	Input Resolution Experiments	19
3.5.4	Data Augmentation Strategies	19
3.5.5	Training Strategy	20
3.5.6	Hyperparameters and Optimizer	21
3.5.7	Comparative Training Setups	21
3.5.8	Challenges and Mitigation	22
3.5.9	Summary	22
3.6	Hardware Setup and Deployment Testing	22
4	ISSUES ENCOUNTERED	25
4.1	Dataset-Related Issues	25
4.2	Training Issues	25
4.3	Model Conversion and Optimization Issues	26
4.4	Hardware Issues (Raspberry Pi 5)	26
4.5	Real-Time Testing Issues (PiCam)	26
4.6	Mitigation Strategies	27
5	RESULTS AND EVALUATION	28
5.1	Training Evaluation	28
5.1.1	Training Dynamics (Loss Curves)	28
5.1.2	Training Performance Metrics	29
5.1.3	Peak Validation Metrics	30
5.1.4	Test Set Evaluation	30
5.2	Deployment on Raspberry Pi 5	31
5.2.1	PyTorch Deployment	31
5.2.2	Quantized TFLite Deployment	31
5.3	Detection Quality	32
5.4	Comparative Analysis	32
6	PROFESSIONAL THESIS	33
6.1	Problem Statement	33
6.2	Literature Review	33
6.3	Methodology	34
6.3.1	Dataset Preparation	34
6.3.2	Model Training	35
6.3.3	Model Optimization	35
6.3.4	Deployment on Raspberry Pi 5	35
6.4	Findings and Discussion	35
6.4.1	Training and Validation	35
6.4.2	Test Set Results	35
6.4.3	Deployment on Raspberry Pi 5	35
6.5	Future Work	36
6.6	Conclusion	37

7 LESSONS LEARNED	38
7.1 Professional Skills	38
7.2 Personal Skills	38
8 CONCLUSION AND PERSPECTIVES	39
Bibliography	39
Annex A – TFLite Conversion Code	41
Annex B – Real-Time Fish Detection Code (Raspberry Pi 5 + PiCam)	42

List of Figures

2.1	Official Logo of the Lab-STICC Laboratory	2
2.2	Map Showing Lab-STICC's Locations Across Brittany.	4
2.3	Organizational Structure of Lab-STICC.	5
2.4	Detailed Organizational Chart for Lab-STICC.	5
3.1	Applications of AI in Marine Science	10
3.2	Underwater Communication Bottleneck	11
3.3	Flowchart of your two-phase YOLOv11 training strategy	21
5.1	Global normalized loss curves (Part 1).	28
5.2	Global normalized loss curves (Part 2).	29
5.3	Evolution of precision, recall, and mAP@0.5 for YOLO variants. (Part 1).	29
5.4	Evolution of precision, recall, and mAP@0.5 for YOLO variants.Part 2	30
5.5	Sample detection outputs with bounding box overlays from PiCam live testing.	32
6.1	Model Compression technique.	34
6.2	Conceptual Future Directions	36

List of Tables

3.1	Comparative training setups for YOLOv11 experiments.	22
3.2	Raspberry Pi 5 test configuration.	23
5.1	Peak validation metrics for YOLO variants.	30
5.2	Test set evaluation metrics for YOLO variants.	31
5.3	PyTorch deployment metrics on Raspberry Pi 5.	31
5.4	Quantized TFLite deployment metrics on Raspberry Pi 5.	31
5.5	Comparative analysis of YOLO variants on Raspberry Pi 5.	32
6.1	Comparison of embedded AI platforms.	34

CHAPTER 1

CONTEXT AND OBJECTIVES

1.1 Context

The internship was conducted at Lab-STICC (UMR CNRS 6285), hosted by the Brest National School of Engineering (ENIB) in Brest, France, from March 2025 to September 2025. This research unit, with over 650 staff members focused on “from sensor to knowledge,” provided an ideal environment for exploring embedded AI applications in marine sciences. The project was a collaboration between the SHAKER team (Software/Hardware and unknown Environment interactions, specializing in optimizing systems under environmental constraints) and the OSE team (Observations, Signal & Environment, emphasizing AI for environmental data analysis).

Underwater drones are essential for marine research, monitoring, and resource management, but face challenges like limited communication, high energy consumption, and computational constraints. Building on prior Lab-STICC work in fish detection and classification, this internship addressed the need for real-time event detection in underwater environments, such as identifying fish, marine debris, or luminosity changes to trigger adaptive algorithms. Public datasets like Fish Recognition Ground-Truth, LifeCLEF 2015 Fish, TrashCan, and Trash-ICRA were used for experiments. Key challenges included technical issues in training deep learning models and balancing performance with resource constraints on embedded platforms.

1.2 Objectives

The primary objective was to design and develop a prototype underwater event detection system using embedded AI, with a focus on real-time object detection for fish and other marine events.

Specific objectives included:

- Conducting a state-of-the-art survey on event detection algorithms in computer vision and developing a custom model using architectures like YOLO or Tiny-YOLO.
- Training and optimizing the deep learning model to overcome technical training problems, achieving good accuracy for real-time fish detection using tools such as OpenCV, YOLOv11 variants, PyTorch, and Raspberry Pi 5.
- Deploying the model on low-power embedded platforms (e.g., NVIDIA Jetson Nano, Raspberry Pi 5 Model B, Beaglebone AI) under Linux-based and real-time OS (e.g., Zephyr), evaluating metrics like power consumption, memory usage, and performance.

This internship directly supports the thesis by providing a practical case study in embedded AI for marine applications, contributing to advancements in environmental monitoring.

CHAPTER 2

ENTERPRISE PRESENTATION

2.1 Overview of the Lab-STICC Research Laboratory

This section provides an overview of Lab-STICC (as shown in Figure 2.1), a key contributor to research in information and communication technologies (ICT) within the Brittany region. We begin by exploring its core identity, historical development, regional presence, and primary research focus areas. Following that, we examine its organizational framework and the various scientific divisions it encompasses. Special emphasis is placed on the AI & Oceans division, with a detailed look at the OSE team (Environmental Observation and Monitoring), including its goals, research priorities, practical applications, selected projects, and collaborative networks.



Figure 2.1: Official Logo of the Lab-STICC Laboratory

2.2 Laboratory Overview

2.2.1 Key Details

- Full Name: Laboratory for Sciences and Technologies in Information, Communication, and Knowledge.
- Short Form: Lab-STICC.
- Designation and Code: CNRS UMR 6285.
- Overseeing Institutions: CNRS (INS2I), Université de Bretagne Occidentale, Université Bretagne Sud, ENSTA Bretagne.
- Primary Location: École Nationale d'Ingénieurs de Brest, Technopôle Brest-Iroise, CS 73862, 29238 BREST Cedex 3, France.

2.2.2 Historical Background

- **2004:** It became evident that the coastal areas of Brittany possessed substantial academic research capabilities in ICT, spread across multiple labs associated with higher education institutions and engineering programs.
- **January 2008:** Initial efforts to consolidate were driven by CNRS and institutional leaders, culminating in the formal establishment of Lab-STICC through mergers and collaborations among research directors.

Today, Lab-STICC serves as a central hub for ICT research in Brittany, centered on the unifying concept: “From sensing devices to informed decision-making: enabling communication and analysis.”

This entity emerged from the integration of four prior groups:

- LEST (Electronics and Telecom Systems Lab),
- TAMCIC (Algorithmic and Hardware Approaches to Communication, Information, and Knowledge Processing),
- LESTER (Real-Time Embedded Systems Electronics Lab),
- SABRES (Statistics Lab of Southern Brittany)
- 2017–2021 Period With around 630 researchers on average, the lab prioritized innovation. Major interdisciplinary initiatives were rolled out during this five-year span, focusing on:
 - Cybersecurity measures,
 - AI advancements,
 - Unmanned aerial vehicles and integrated systems,
 - Robotic technologies,
 - Immersive virtual and augmented realities.

2.3 Presentation of the Laboratory

2.3.1 Regional Distribution

Lab-STICC operates across various locations in Brittany (refer to Figure 2.2), enhancing opportunities for local partnerships and joint projects:

- Brest Site: Includes IMT Atlantique, ENSTA Bretagne, ENIB, and Université de Bretagne Occidentale campuses.
- Quimper Site: Hosted at the IUT de Quimper under Université de Bretagne Occidentale.
- Lorient and Vannes Sites: Affiliated with Université Bretagne Sud.

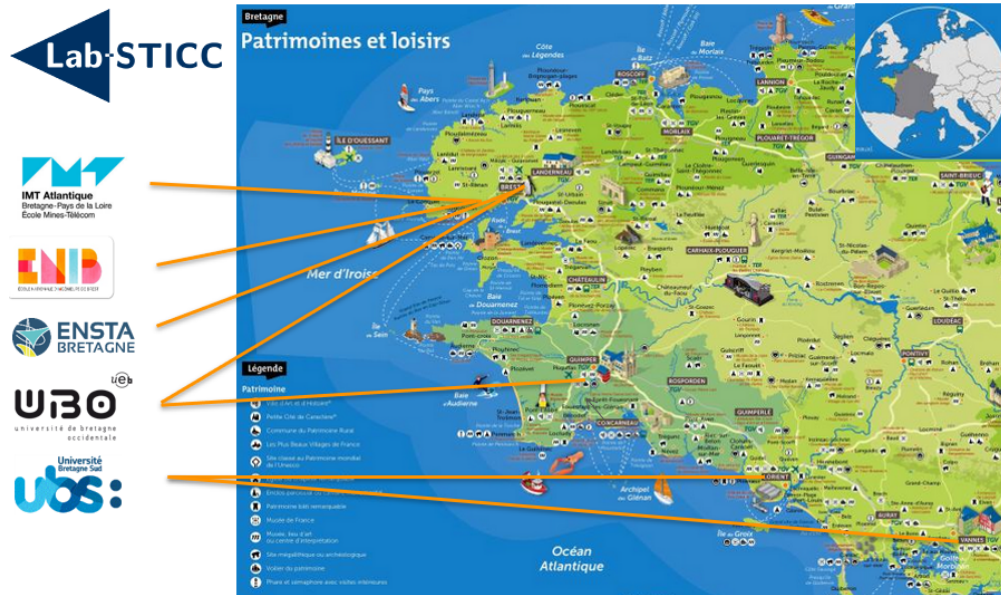


Figure 2.2: Map Showing Lab-STICC's Locations Across Brittany.

2.3.2 Core Operations and Stakeholders

- **Primary Research Domains:** Lab-STICC engages in studies spanning information theory, electromagnetic waves and materials, onboard electronics and computing, data analytics and AI, signal processing and communication, user interfaces, robotics, and digital security.
- **Practical Sectors:** Applications extend to ocean monitoring, IoT devices, defense systems, aerospace, healthcare, safety protocols, and autonomous robotics.

2.3.3 Organizational Framework

Lab-STICC is divided into 9 main divisions (depicted in Figure 2.3), housing 26 specialized research groups. Each division oversees scientific alignment and unity among its groups, while the groups handle targeted research initiatives based on their expertise (as outlined in Figure 2.4):

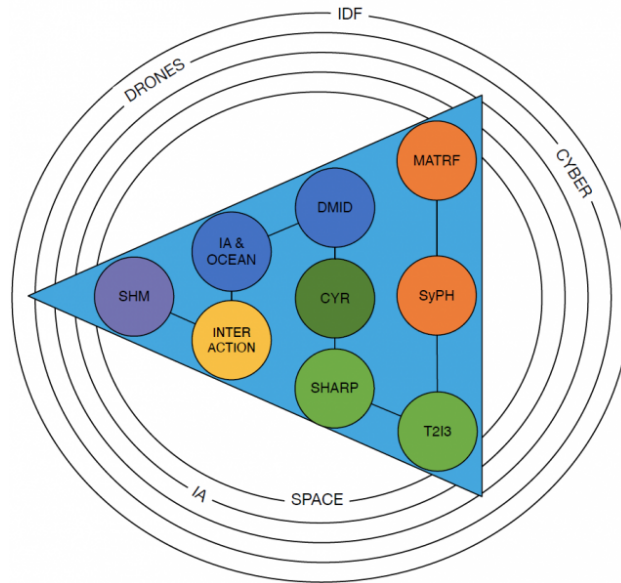


Figure 2.3: Organizational Structure of Lab-STICC.

- MatRF (Materials and Radio Frequency).
- SyPH (Photonics and High-Frequency Systems).
- T2I3 (Information Processing, Transmission, Algorithms, and Integration).
- DMD (Data, Models, Information, and Decision-Making).
- CyR (Cybersecurity and Networking).
- SHARP (Software, Hardware, Architectures, and Processes).
- AI & Oceans.
- INTERACTION.
- SHM (Human-Machine Interaction Systems).

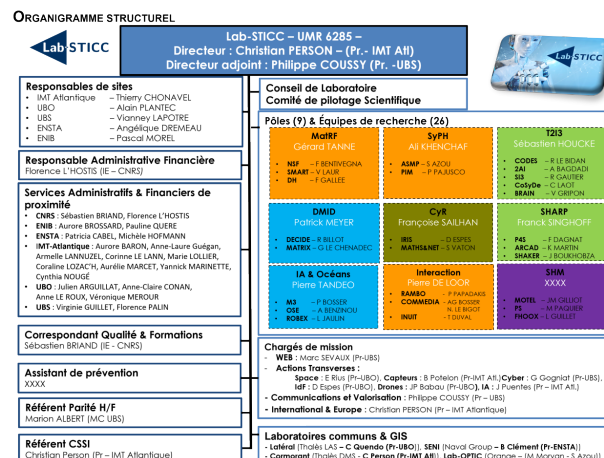


Figure 2.4: Detailed Organizational Chart for Lab-STICC.

2.4 Presentation of the Host Department

2.4.1 AI & Oceans Division

The primary objective of the AI & Oceans division is to enhance the expertise and visibility of Lab-STICC at national and international levels by positioning its research at the interface of artificial intelligence and marine and ocean sciences. This division views the ocean as a complex system, integrating physical, biological, ecological interactions, as well as human activities (maritime traffic, coastal management, exploitation of marine resources, etc.).

It is organized around three specific research areas:

- AI and marine environment perception (ROBEX, OSE, M3),
- AI and exploratory robotics (ROBEX),
- AI and oceanic and marine data (M3, OSE).

The division's work is distinguished by strong interdisciplinarity, combining theoretical contributions in artificial intelligence with their application to marine environmental issues. These studies are conducted in collaboration with other scientific and technological disciplines, such as offshore technologies, space observation, physical oceanography, hydrography, marine ecology, and maritime surveillance.

The OSE Team

The OSE (Observation and Surveillance of the Environment) team focuses its research on applying artificial intelligence (AI) to the analysis of massive environmental data from remote sensing (satellites, beacons, underwater videos, etc.). Facing the explosion of such data, the goal is to develop robust, explainable learning methods that incorporate physical knowledge. The work is structured around three main areas: hybrid AI-physical models, algorithm robustness, and solving inverse problems.

- **Team Name:** Observations, Signal & Environment (OSE).
- **Division:** Artificial Intelligence & Oceans.
- **Scientific Focus:** Development of AI methods applied to marine environmental observation, with an emphasis on multimodal remote sensing (satellite imagery, GNSS data, AIS, passive acoustics, underwater videos).
- **Objectives:**
 - * Apply AI to environmental data analysis to support societal stakeholders in addressing climate and biodiversity challenges.
 - * Integrate physical knowledge into learning models.
 - * Enhance algorithm robustness for rare, incomplete, or highly variable data.
 - * Solve environmental inverse problems (interpolation, signal inversion, information reduction, etc.).

- **Application Areas:**
 - * Oceanic remote sensing (satellite).
 - * Aerial and maritime monitoring (GNSS, ARGOS, AIS, acoustics).
 - * Analysis of underwater videos.
- **Examples of Work:**
 - * Learning variational models for reconstructing oceanic geophysical dynamics.
 - * Weighting climate simulations using AI methods.
 - * Deep learning for automatic 3D identification of benthic species.
 - * Cetacean tracking via passive acoustics.
 - * Fish species recognition in underwater videos.
- **Industrial Collaborations:** Thales, NavalGroup, CLS, Eodyn, Actimar, Hytech-imaging.
- **Institutional Partners:** CNES, DGA, IFREMER, IUEM, INRIA, Météo France, Mercator Ocean, IRISPACE, EUR ISblue, SHOM, FEM.
- **Academic Partnerships:** Sorbonne University, UCLA, University of Washington, ETH Zurich, University Laval, University of Buenos Aires, RIKEN, etc.

CHAPTER 3

MISSION WITH ENTERPRISE

3.1 Introduction to the Mission

The mission of this internship was to design and implement an underwater event detection system based on embedded artificial intelligence (AI), with a specific focus on deploying deep learning models onto the Raspberry Pi 5 platform. The core objective was to enable real-time detection of underwater events, such as the presence of fish or other marine objects, under strict resource constraints. This problem is highly relevant to marine sciences and robotics, where autonomous platforms such as underwater drones and remotely operated vehicles (ROVs) are increasingly used for environmental monitoring, biodiversity assessment, and marine resource management.

Unlike terrestrial computer vision applications, underwater event detection presents a unique set of scientific and technical challenges. Underwater imagery is inherently complex due to environmental distortions. Light absorption and scattering reduce visibility and alter colors, often producing images dominated by blue or green tones. Turbidity, caused by suspended particles, introduces blur and noise, while marine organisms frequently appear partially occluded by plants, rocks, or other animals. These conditions reduce the effectiveness of standard image recognition models trained on terrestrial datasets.

From an engineering perspective, deploying AI models in marine environments requires low-power embedded hardware. Platforms like the Raspberry Pi 5 offer portability and affordability, but they lack the computational capabilities of high-end GPUs. Running state-of-the-art deep learning models under these conditions requires careful optimization, including model size reduction, efficient memory management, and trade-offs between accuracy and inference speed. Thus, the mission sits at the intersection of two research domains: underwater computer vision and embedded AI for resource-constrained systems.

3.1.1 Challenges in Underwater AI Event Detection

The broader research community has long recognized the challenges associated with applying AI to underwater monitoring tasks. Several issues are consistently highlighted:

- **Image quality degradation:** Unlike air, water absorbs and scatters light much more strongly, especially at longer wavelengths (reds and yellows). As a result, underwater images often suffer from color shifts, reduced contrast, and low signal-to-noise ratios.

- **Dynamic environments:** Moving particles, floating debris, and constantly changing lighting conditions create visual clutter, complicating the detection of small objects such as fish.
- **Dataset limitations:** Annotated underwater datasets remain relatively scarce compared to terrestrial datasets like COCO or ImageNet. Furthermore, existing datasets often focus on specific species or regions, limiting model generalizability.
- **Hardware constraints:** While high-performance models achieve strong accuracy in laboratory conditions, transferring them to low-power platforms such as the Raspberry Pi remains an open challenge.

3.1.2 Existing Approaches and Related Work

Recent years have seen growing interest in applying computer vision to marine applications. Several datasets and detection frameworks have been proposed:

- **Fish Recognition Ground-Truth (FRGT) and LifeCLEF Fish datasets** provide annotated fish images collected from underwater cameras, enabling research on species classification and detection.
- **Fish4Knowledge** is another large-scale dataset focusing on video analysis of coral reef environments, widely used for benchmarking fish detection algorithms.
- **TrashCan and Trash-ICRA datasets** target underwater waste detection, addressing pollution monitoring and marine debris recognition.

On the algorithmic side, multiple families of models have been evaluated for underwater tasks:

- **Two-stage detectors**, such as Faster R-CNN, achieve high accuracy but are computationally heavy, making them unsuitable for embedded deployment.
- **Single-stage detectors**, such as SSD (Single Shot MultiBox Detector), are faster but struggle with small object detection, which is critical in fish recognition tasks.
- **YOLO-based models** (You Only Look Once), particularly recent versions such as YOLOv5, YOLOv8, and now YOLOv11, provide an excellent compromise between accuracy and efficiency. Several studies have shown YOLO's effectiveness in detecting fish, marine debris, and even plankton in underwater environments. However, most of these implementations rely on GPU-powered systems.

Despite these advances, few works focus specifically on real-time deployment on embedded devices. The majority of studies demonstrate results using high-performance computing infrastructures, leaving a significant gap between research prototypes and field-ready systems. This gap is precisely where this internship project positions itself.

3.1.3 Mission Reframing

Given these challenges and research gaps, the mission can be reframed as follows:

- **Scientific Contribution:** Explore the adaptation of modern object detection architectures (YOLOv11) to the underwater domain, accounting for environmental distortions and dataset limitations.
- **Engineering Contribution:** Develop and deploy an optimized version of the detection model on the Raspberry Pi 5, balancing accuracy, inference speed, and energy efficiency.

Therefore, this internship project focuses on bridging the gap between state-of-the-art underwater detection research and practical embedded deployment, with the Raspberry Pi 5 serving as a testbed for real-time marine AI applications.

3.2 Project and Industry Context

The increasing reliance on autonomous systems in marine environments reflects a global need to better monitor, understand, and preserve ocean ecosystems. Autonomous Underwater Vehicles (AUVs), Remotely Operated Vehicles (ROVs), and underwater drones have become essential tools for marine biology, resource management, defense applications, and pollution monitoring. These platforms enable the collection of critical environmental data while reducing the risks and costs associated with human-operated missions in deep or hazardous waters. turbidity).

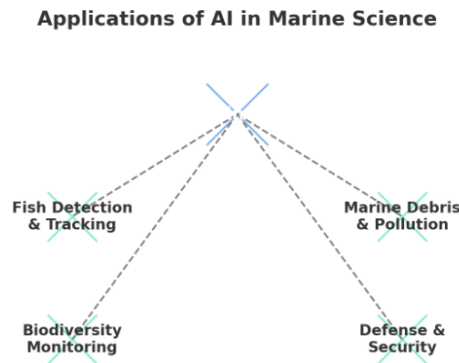


Figure 3.1: Applications of AI in Marine Science

3.2.1 Industry Relevance of AI in Marine Applications

Several industrial and research domains are driving the adoption of embedded AI in underwater systems:

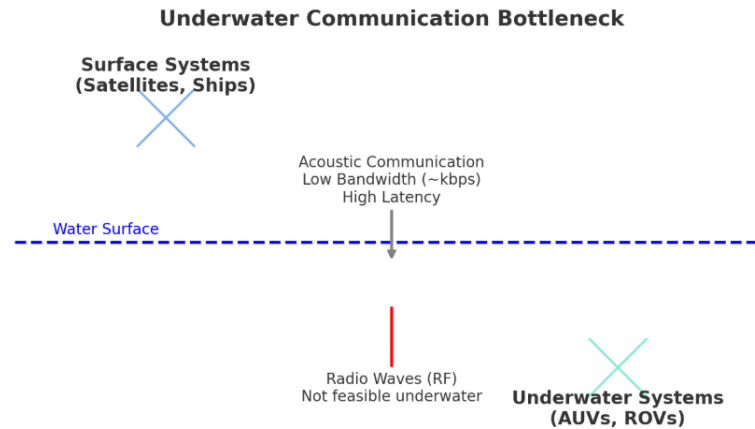


Figure 3.2: Underwater Communication Bottleneck

- **Environmental Monitoring and Biodiversity Conservation:** Organizations like IFREMER (France), NOAA (USA), and UNESCO’s Ocean Decade initiatives increasingly rely on AUVs to monitor ecosystems. AI enables automatic fish recognition, species distribution mapping, and detection of invasive species.
- **Pollution and Waste Detection:** With global attention on plastic pollution, AI-based object detection systems such as those trained on TrashCan and Trash-ICRA datasets help identify underwater debris. This information is crucial for supporting cleanup operations and policy decisions.
- **Defense and Security:** Naval applications demand real-time detection of mines, submarines, and anomalous activities. Embedded AI enables stealthy, autonomous, and prolonged missions without constant operator supervision.
- **Fisheries and Resource Management:** Fish detection and tracking allow for non-intrusive stock assessment, supporting sustainable fishing practices. AI-based tools reduce reliance on manual data analysis, saving both time and cost.

3.2.2 Current Technological Gaps

Despite the growing relevance, several technological bottlenecks remain:

- **Communication limitations:** Underwater communication relies on acoustic signals rather than radio waves, offering very limited bandwidth. This prevents continuous transmission of high-resolution video to surface operators and demands on-board processing.
- **Energy constraints:** Long-duration AUV missions are limited by battery life. Running large AI models increases energy consumption significantly, which reduces the mission duration.
- **Computational constraints:** Many high-performing detection models require GPUs with large memory, but AUVs typically carry only lightweight embedded

hardware. Running real-time inference on platforms like Raspberry Pi remains a key challenge.

- **Data limitations:** While datasets such as LifeCLEF Fish and Fish4Knowledge exist, they are still relatively small compared to terrestrial datasets like COCO or ImageNet. Moreover, they often lack the diversity needed to generalize across global ocean conditions.

3.2.3 Positioning of This Project

This internship project directly addresses these gaps by deploying an AI-based underwater event detection system on Raspberry Pi 5, a compact and low-cost embedded platform. Unlike traditional approaches that require powerful GPU clusters, this project demonstrates that real-time inference for fish detection is achievable on affordable and portable hardware.

By focusing on Raspberry Pi 5, the project contributes to the development of scalable and field-deployable solutions that can be integrated into AUVs, ROVs, or stationary underwater monitoring systems. This aligns with the strategic objectives of the AI & Oceans Division of Lab-STICC, which aims to bridge the gap between AI research and marine applications.

Furthermore, this work contributes to ongoing initiatives such as the ISblue program and France 2030 investment plan, both of which prioritize sustainable ocean monitoring technologies. It also aligns with international efforts like the United Nations Decade of Ocean Science for Sustainable Development (2021–2030), which emphasizes the role of digital and AI-driven tools in preserving marine biodiversity.

3.3 My Role and Responsibilities

As an Embedded AI Intern at Lab-STICC (UMR CNRS 6285), my primary responsibility was to design, develop, and deploy a real-time underwater event detection system based on YOLOv11 models, optimized for execution on the Raspberry Pi 5 platform. This role required balancing advanced machine learning research with practical engineering for embedded deployment.

My responsibilities can be organized into the following categories:

3.3.1 Literature Review and Benchmarking

The first stage of my work was conducting a comprehensive review of state-of-the-art algorithms in computer vision for underwater object detection. I compared the following approaches:

- **Two-stage detectors** (e.g., Faster R-CNN): strong accuracy but unsuitable for embedded deployment due to high latency.

- **Single-stage detectors** (e.g., SSD, EfficientDet): computationally efficient but poor performance on small fish objects.
- **YOLO-based architectures**: found to offer the best compromise between accuracy, speed, and scalability.

This benchmarking justified the choice of YOLOv11 for the internship. I also studied related datasets such as Fish Recognition Ground-Truth, LifeCLEF Fish, and Trash-Can, which highlighted the importance of preprocessing and augmentation for underwater imagery.

3.3.2 Dataset Preparation and Preprocessing

I was responsible for curating and preparing datasets for training. This involved:

- **Data collection**: consolidating samples from Fish Recognition Ground-Truth and LifeCLEF Fish datasets.
- **Annotation handling**: adapting existing bounding box annotations to YOLO format.
- **Preprocessing**: resizing images to 640×640 pixels, normalizing pixel values, and applying underwater-specific augmentations (brightness correction, noise injection, random flips, and hue adjustments).
- **Dataset balancing**: ensuring minority classes (rare species or smaller fish) were not underrepresented.

This preprocessing phase was crucial for building robust models capable of handling low-light and turbid conditions.

3.3.3 Algorithm Development and Training

I developed custom training pipelines using PyTorch 2.4.1 and the Ultralytics YOLOv11 library on an NVIDIA A40 GPU provided by the laboratory. Key responsibilities included:

- **Model selection**: experimenting with YOLOv11n and YOLOv11s variants.
- **Progressive augmentation**: introducing staged data augmentation to improve generalization.
- **Hyperparameter tuning**: optimizing batch size, learning rate, and weight decay to balance convergence speed and accuracy.
- **Evaluation**: monitoring precision, recall, and mAP@0.5 across epochs.

The final models achieved mAP scores between 0.82 and 0.88 on validation datasets, with inference times suitable for real-time testing on embedded platforms.

3.3.4 Model Optimization for Raspberry Pi 5

A significant part of my role was ensuring that trained models could be deployed on the Raspberry Pi 5, which has:

- ARM-based CPU (2.4 GHz quad-core).
- 8 GB RAM.
- No native CUDA GPU support.

My tasks included:

- Exporting YOLO models to ONNX format for compatibility.
- Testing quantization strategies (float16, int8) to reduce model size and memory footprint.
- Running inference benchmarks using live camera input connected to the Raspberry Pi.
- Measuring FPS, CPU utilization, and power consumption under different configurations.

This stage required troubleshooting installation issues (ARM64 support, dependencies) and optimizing Python scripts for low-latency performance.

3.3.5 Evaluation and Result Analysis

I was responsible for evaluating the system both in simulation and on hardware:

- **Training evaluation:** precision, recall, confusion matrices.
- **Embedded evaluation:** real-time FPS, latency per frame, and detection quality under different lighting and water clarity.
- **Comparisons:** YOLOv11n vs YOLOv11s, highlighting trade-offs between accuracy and speed.

These evaluations allowed me to identify bottlenecks and suggest improvements for future development (e.g., using Coral TPU or Jetson platforms).

3.3.6 Collaboration and Reporting

Throughout the internship, I worked closely with researchers from the OSE team (Observations, Signal & Environment) and the SHAKER team (Software/Hardware and unKnown Environment inteRactions). My responsibilities included:

- Attending weekly meetings to discuss progress and challenges.

- Documenting experimental procedures and results.
- Contributing to internal project presentations.
- Preparing this report as a formal documentation of my internship outcomes.

3.3.7 Summary

In summary, my role combined research-oriented tasks (literature review, dataset study, algorithm benchmarking) with engineering-oriented tasks (model deployment, Raspberry Pi optimization, benchmarking). This dual approach ensured that the project not only advanced scientific understanding of underwater detection but also delivered a practical prototype suitable for real-world applications.

3.4 Dataset Preparation and Preprocessing

The preparation of the dataset forms the foundation of any computer vision project. In underwater event detection, dataset preparation is even more critical due to the unique challenges posed by the marine environment, such as image distortion, turbidity, poor illumination, and occlusion. Without careful handling of the dataset, trained models tend to overfit to noise or fail to generalize to realistic conditions.

For this project, the primary dataset used was the Fish Recognition Ground-Truth (FRGT) dataset, which consists of underwater images containing fish captured in diverse environmental conditions. Since the internship's focus was on developing a real-time fish detection system, the dataset was simplified to a single-class detection problem ("fish"). The following subsections describe in detail the preparation and preprocessing steps applied.

3.4.1 Importance of Dataset Preparation in Underwater AI

Unlike terrestrial datasets such as COCO or ImageNet, underwater datasets are relatively small, domain-specific, and often lack consistency in annotations. Furthermore, underwater images contain distortions that make them very different from images captured in air. This means that preparing the dataset is not just a mechanical step but a scientific necessity to ensure the model can learn robust features.

- **Illumination Issues:** Light attenuates quickly underwater, especially at red wavelengths, leading to predominantly blue-green images.
- **Motion and Turbidity:** Floating particles and marine currents add blur or occlusion.
- **Limited Data Volume:** Unlike millions of terrestrial images, underwater datasets typically contain only thousands of images.

Therefore, dataset preparation for this project focused on quality control, annotation accuracy, and preprocessing pipelines tailored to underwater environments.

3.4.2 Dataset Labeling

The FRGT dataset did not originally provide YOLO-compatible annotations. Therefore, one of my first tasks was to create bounding box labels for each image. Using labeling tools (e.g., LabelImg), I manually drew boxes around fish in the images. Each annotation was exported in the YOLO format, where each bounding box is described by:

`<class_id> <x_center> <y_center> <width> <height>`

Since this project is a single-class problem, the `class_id` was always 0 (representing “fish”).

All bounding box coordinates were normalized relative to the image size, ensuring compatibility with the YOLO training pipeline.

This process required significant effort, as manual labeling is time-consuming but essential for producing reliable training data.

3.4.3 Dataset Cleaning

During labeling and review, I identified multiple issues that could reduce dataset quality if left unaddressed. These included:

- **Corrupted images:** Some files could not be opened or displayed. These were removed.
- **Images with no visible fish:** Such images were excluded, as they provided no meaningful supervision signal for the detector.
- **Mislabeled bounding boxes:** Some bounding boxes were initially too large, too small, or misplaced due to manual error. These were corrected after visual inspection.

Cleaning the dataset improved consistency and ensured that every sample contributed positively to the learning process.

3.4.4 Input Resolution Variations

One of the key aspects of dataset preparation for this project was testing different input image resolutions. YOLO allows dynamic resizing through the `imgsz` parameter. I experimented with both:

- **640 × 640 pixels:**
 - * *Advantages:* Lower memory requirements and faster inference.
 - * Used mainly with YOLOv11n (nano) models for lightweight testing and deployment.
- **800 × 800 pixels:**
 - * *Advantages:* Higher resolution retains more details, improving small-object detection (important for small fish in cluttered environments).
 - * *Disadvantages:* Higher computational cost, slower inference on embedded devices.
 - * Used mainly with YOLOv11s (small) models during high-accuracy experiments.

These experiments highlighted a fundamental trade-off in embedded AI: accuracy vs. efficiency. Higher resolutions produced better detection results, but at the expense of speed, which is critical for real-time deployment on Raspberry Pi 5.

3.4.5 Normalization and Standardization

As part of the preprocessing pipeline, the Ultralytics YOLO framework automatically performed:

- **Pixel normalization:** Converting image pixel values from $[0 - 255]$ to $[0 - 1]$.
- **Bounding box normalization:** Scaling bounding box coordinates to relative values (fractions of image width/height).

These standardizations ensured stable gradient computations and prevented numerical instability during training.

3.4.6 Challenges in Dataset Preparation

Several challenges arose during dataset preparation:

- **Manual labeling overhead:** Annotating hundreds of images required significant time and concentration. Small errors in bounding boxes could have reduced model accuracy.
- **Low-visibility images:** Some images contained fish partially hidden by rocks, algae, or low-light conditions, making labeling difficult.
- **Balancing detail and efficiency:** Choosing between 640×640 and 800×800 input sizes required careful consideration of model accuracy vs. inference performance on Raspberry Pi.

Despite these challenges, a consistent and well-structured dataset was produced, enabling robust model training.

3.5 Algorithm Development and Training

Once the dataset was fully prepared, the next stage involved designing and training the object detection models. This step was at the heart of the internship mission, since the goal was to produce a detection pipeline that was both accurate in recognizing fish and efficient enough to run on embedded hardware (Raspberry Pi 5). To achieve this, I relied on the Ultralytics YOLOv11 framework, exploring multiple model variants, resolutions, augmentation strategies, and training schedules.

The subsections below provide an in-depth explanation of the algorithm development process, justifications for each choice, and observations drawn from the experiments.

3.5.1 Rationale for Choosing YOLOv11

The YOLO family of architectures (You Only Look Once) has become the benchmark for real-time object detection. Unlike two-stage detectors such as Faster R-CNN, which separate region proposal and classification into sequential stages, YOLO performs detection in a single forward pass. This enables near real-time inference, which is crucial for time-sensitive applications such as marine robotics.

Among the available YOLO variants, YOLOv11 (released in 2024) was selected for the following reasons:

- Improved backbone and neck design compared to earlier YOLO versions, allowing better detection of small and partially occluded objects.
- Native support for modern augmentation strategies (Mosaic, MixUp, HSV adjustments) that are particularly useful for noisy underwater imagery.
- Scalable architecture: YOLOv11 comes in multiple variants (nano, small, medium, large), enabling flexible experimentation depending on available hardware resources.
- Compatibility with Ultralytics training pipeline, which provides easy integration, automatic logging, and ready-to-use pretrained weights.

Alternative architectures such as Faster R-CNN and SSD were considered but rejected due to either high computational demand (Faster R-CNN) or relatively lower accuracy on small-object detection (SSD). Since the ultimate deployment target was Raspberry Pi 5, a lightweight yet accurate YOLO variant was the logical choice.

3.5.2 Model Variants Explored

I trained and compared multiple YOLOv11 variants to understand the trade-offs between detection accuracy, inference speed, and resource usage.

- **YOLOv11n (Nano)**

- * Contains the fewest parameters and smallest architecture.
- * Extremely fast on Raspberry Pi 5, making it ideal for edge deployment.
- * *Limitation*: lower accuracy, especially on small fish in cluttered environments.
- **YOLOv11s (Small)**
 - * Larger than YOLO-n but still lightweight.
 - * Provides significantly better accuracy while maintaining reasonable speed.
 - * A practical balance for real-world embedded AI applications.
- **Hybrid Strategy (YOLO-n + YOLO-s)**
 - * I experimented with training strategies where YOLO-s inherited certain weights or configurations from YOLO-n.
 - * The goal was to achieve a “middle ground”: fast inference like YOLO-n but with higher precision closer to YOLO-s.

This exploration demonstrated how architectural choices influence performance. While YOLO-n was best for speed, YOLO-s consistently delivered higher precision.

3.5.3 Input Resolution Experiments

Input resolution plays a critical role in detection performance. Larger inputs preserve more fine-grained details but increase memory and computation requirements.

- **640 × 640 pixels**
 - * Used in baseline experiments.
 - * Lower memory usage, faster training and inference.
 - * Ideal for lightweight deployment scenarios.
- **800 × 800 pixels**
 - * Preserved more details, especially useful for small fish or distant detections.
 - * Improved mean Average Precision (mAP).
 - * Heavier on GPU/CPU, resulting in slower inference.

By systematically testing these two resolutions, I was able to quantify the accuracy–speed trade-off, a central consideration for embedded deployment.

3.5.4 Data Augmentation Strategies

Augmentation is especially important in underwater AI, where conditions such as lighting, turbidity, and occlusion vary dramatically.

- **Standard Augmentation (Baseline YOLO Pipeline)**
 - * Horizontal flips ($p = 0.5$): Simulates fish swimming in both directions.

- * Brightness and saturation shifts (HSV adjustments): Mimics varying lighting conditions underwater.
- * Scaling and translations: Helps the model generalize to fish appearing at different positions.
- * Mosaic & MixUp: Creates composite images, improving robustness to clutter.
- **Progressive Augmentation (Custom Strategy)**
 - * To enhance generalization, I introduced a progressive augmentation schedule:
 - * Epochs 0–100 → Strong augmentation
 - Large HSV changes, mosaic probability of 0.35, mixup enabled.
 - Forces the model to learn under difficult visual conditions.
 - * Epochs 101–200 → Medium augmentation
 - Reduced HSV changes, mosaic probability decreased.
 - Stabilizes training while maintaining variety.
 - * Epochs 201–300 → Light augmentation
 - Minimal augmentation.
 - Allows the model to refine predictions without heavy distortions.
 - * Fine-tuning Phase → Very light augmentation
 - Almost clean images, only slight color jitter.
 - Encourages precision in final model weights.

This progressive strategy proved particularly useful because it mirrored the curriculum learning principle — training the model with harder examples first, then gradually simplifying the task.

3.5.5 Training Strategy

Training was organized into two main phases:

- **Phase 1: Transfer Learning with Frozen Backbone**
 - * Started with pretrained YOLOv11 weights.
 - * Froze early backbone layers (`freeze=5`), which capture generic visual patterns.
 - * Trained mainly the detection head and higher layers.
 - * Applied progressive augmentation during this phase.
- **Phase 2: Fine-Tuning (Unfrozen Network)**
 - * Loaded the best checkpoint from Phase 1.
 - * Unfroze all layers (`freeze=0`).
 - * Reduced augmentation strength to near-zero.
 - * Lower learning rate (`lr=0.001`) for stable refinement.
 - * Trained an additional 100 epochs for fine-grained optimization.

This two-step process ensured efficient use of pretrained knowledge while tailoring the network to the specific underwater dataset.

Two-Phase YOLOv11 Training Strategy

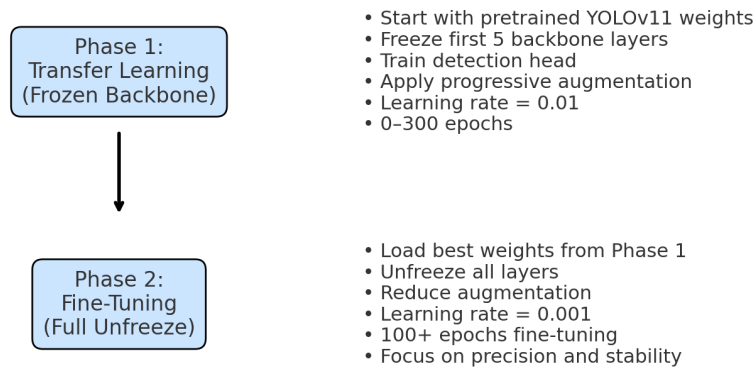


Figure 3.3: Flowchart of your two-phase YOLOv11 training strategy

3.5.6 Hyperparameters and Optimizer

- **Optimizer:** Stochastic Gradient Descent (SGD) was selected for its stability in large-scale training. Although Adam and AdamW are alternatives, SGD with momentum tends to generalize better.
- **Learning Rates:**
 - * Initial: $1r0=0.01$ for rapid adaptation during Phase 1.
 - * Fine-tuning: $1r0=0.001$ for careful convergence.
- **Momentum:** 0.937, to accelerate convergence and escape shallow minima.
- **Weight Decay:** 0.0005 to reduce overfitting.
- **Batch Size:** 32, balancing memory use and gradient stability.
- **Early Stopping Patience:** 50 epochs, preventing wasted computation on stalled models.

These hyperparameters were tuned experimentally, ensuring that models converged stably while avoiding overfitting.

3.5.7 Comparative Training Setups

To systematically evaluate the influence of different factors, I organized the experiments as follows:

This structured experimentation allowed me to clearly see the trade-offs between speed, accuracy, and computational cost.

Model	Input Size	Augmentation	Training	Key Notes
YOLOv11n	640×640	Standard	End-to-end	Fastest, lowest accuracy
YOLOv11n	640×640	Progressive	Two-phase	Robust, lightweight
YOLOv11s	800×800	Standard	End-to-end	Higher accuracy, slower
YOLOv11s	800×800	Progressive	Two-phase	Best balance, robust
Hybrid (n+s)	Mixed	Progressive	Two-phase	Balanced, experimental

Table 3.1: Comparative training setups for YOLOv11 experiments.

3.5.8 Challenges and Mitigation

During training, I encountered several challenges:

- **Overfitting on small dataset:** Mitigated using strong augmentation and weight decay.
- **GPU memory limitations:** Solved by reducing batch size or using 640 resolution.
- **Unstable training at high augmentations:** Addressed through progressive scheduling.
- **Slow inference on larger models:** Balanced by testing YOLO-n vs YOLO-s.

Each of these challenges provided practical insights into the realities of training embedded AI models for underwater tasks.

3.5.9 Summary

The algorithm development process involved:

- Selecting YOLOv11 for its balance of speed and accuracy.
- Experimenting with YOLO-n, YOLO-s, and hybrid configurations.
- Testing multiple input resolutions (640, 800).
- Designing a custom progressive augmentation schedule.
- Implementing a two-phase training strategy with transfer learning and fine-tuning.
- Comparing setups to analyze trade-offs between accuracy and efficiency.

3.6 Hardware Setup and Deployment Testing

This subsection details the physical hardware environment and the systematic methodology used to test the deployed models on the target device, a Raspberry Pi 5. This

ensures all performance results are measurable, reproducible, and accurately reflect the potential for real-world deployment.

Target Hardware Specification

The models were deployed and tested on a Raspberry Pi 5, representing a modern, mid-range single-board computer capable of handling more complex edge AI workloads. The key specifications are outlined below:

Component	Specification
SoC	Broadcom BCM2712
CPU	Quad-core Arm Cortex-A76 @ 2.4GHz
RAM	4GB LPDDR4X-4267
Storage	32GB SanDisk Extreme MicroSD Card (A2, V30)
OS	Raspberry Pi OS (64-bit), Linux kernel 6.6
Python	Version 3.11.2
Power	Official Raspberry Pi 27W USB-C Power Supply

Table 3.2: Raspberry Pi 5 test configuration.

Rationale for Choice: The Raspberry Pi 5 was selected as the target deployment platform due to its increased processing power over previous generations, making it a suitable candidate for evaluating the real-time performance of object detection models. Its balance of performance, energy efficiency, and cost is highly relevant for practical embedded vision applications.

Software Environment and Dependencies

A minimal and optimized software stack was configured to run the models efficiently and minimize overhead.

- **Core Framework:** TensorFlow Lite Runtime 2.15.0
- **Acceleration Library:** The XNNPACK delegate for TFLite was enabled to fully utilize the Pi 5’s CPU capabilities for accelerated floating-point and quantized inference.
- **Supporting Libraries:** OpenCV-Python (for image loading and preprocessing) and NumPy.
- **Environment Setup:** A dedicated virtual environment was created using `venv` to ensure dependency isolation and version control.

Deployment and Benchmarking Methodology

A standardized and automated benchmarking pipeline was developed to ensure consistent and fair evaluation across all model variants.

1. Model Conversion Pipeline:

The journey from a trained model to a deployed one followed these steps:

- **Input:** Best pre-trained .pt weights from training.
- **Step 1:** Export to TFLite. The Ultralytics YOLO export API was used (`model.export(format='tflite')`) to generate a full-integer quantized (INT8) model. A representative dataset was used to calibrate the quantization thresholds.
- **Output:** Optimized .tflite model file ready for deployment.

2. Performance Benchmarking Script:

A custom Python script (`litetf.py`) was created to perform the following tasks automatically:

- **Warm-up:** Run the model on a few dummy inputs to initialize the TFLite interpreter and delegate.
- **Timing Loop:** Perform inference on a curated set of 400 images from the test set.
- **Metrics Collection:**
 - * **Inference Time:** Measured per image using `time.perf_counter()`, capturing only the core `interpreter.invoke()` call.
 - * **Frames Per Second (FPS):** Calculated as $1/(\text{average_inference_time_in_seconds})$.
 - * **CPU Utilization:** Sampled using the `psutil` library.
 - * **Memory Footprint:** Recorded the change in resident set size (RSS) memory usage.
 - * **Statistical Analysis:** Computed the average, minimum, maximum, and standard deviation for inference time.

3. Evaluation of Accuracy:

The final quantized TFLite models were also evaluated for accuracy using a modified validation script to ensure the quantization process did not lead to a significant drop in mAP on the full test set.

Conclusion of Deployment Methodology

This rigorous methodology for deployment testing on the Raspberry Pi 5 ensures that all reported performance metrics for inference speed, computational efficiency, and model accuracy are collected in a consistent and reliable manner. The results provide a critical, real-world benchmark for evaluating the deployability of each model variant.

CHAPTER 4

ISSUES ENCOUNTERED

During the internship, several issues were encountered at different stages of the project, ranging from dataset preparation to model training, optimization, and deployment on embedded hardware. These challenges are natural in research-oriented work, especially when dealing with underwater datasets and resource-constrained platforms such as the Raspberry Pi 5. Documenting these problems is important, as they shaped the technical decisions, optimizations, and final outcomes of the system.

4.1 Dataset-Related Issues

The dataset used for this project consisted of images extracted directly from underwater video recordings. While this approach allowed the collection of a large number of samples, it also introduced specific challenges:

- **Frame redundancy:** Consecutive frames from the same video often contained nearly identical fish poses and positions. This redundancy reduced the effective diversity of the dataset and increased the risk of overfitting.
- **Blurry frames:** Motion blur caused by rapid fish movements and underwater turbidity was common. Several extracted frames were visibly blurred, which made feature extraction more difficult for the model.
- **Occlusions:** Fish were sometimes partially hidden behind algae, rocks, or other marine organisms. These occlusions complicated both manual labeling and model training, as bounding boxes had to be approximated.

Although cleaning steps were applied to remove corrupted or empty images, the issues above were inherent to the source data and could not be fully eliminated.

4.2 Training Issues

Training deep learning models for underwater detection also posed challenges:

- **GPU memory limits:** Training larger YOLOv11 variants (e.g., YOLOv11s at 800×800 resolution) often exceeded available GPU memory, forcing reductions in batch size or input resolution.
- **Overfitting:** Due to the limited diversity in the dataset, models occasionally achieved very high accuracy on training data but failed to generalize well on validation data. This was mitigated by applying strong data augmentation strategies and weight regularization.

- **Unstable convergence:** When heavy augmentations (e.g., strong mosaic or HSV shifts) were applied early in training, the models sometimes failed to converge smoothly. A progressive augmentation schedule was therefore adopted to stabilize learning.

4.3 Model Conversion and Optimization Issues

To deploy the trained YOLOv11 models on embedded hardware, conversion to lighter formats was necessary. However, this process introduced multiple technical problems:

- **ONNX and TensorFlow Lite conflicts:** Converting YOLOv11 models first into ONNX and then into TensorFlow Lite caused dependency mismatches between libraries such as NumPy, `protobuf`, and `flatbuffers`. These incompatibilities frequently broke the conversion pipeline.
- **Quantization challenges:** Although quantization (FP16 and INT8) successfully reduced model size, it sometimes degraded detection accuracy beyond acceptable limits. Achieving a balance between efficiency and performance required extensive testing.
- **Compatibility issues:** Some TensorFlow Lite builds did not fully support ARM64 architecture on Raspberry Pi, which required downgrading or rebuilding specific packages manually.

4.4 Hardware Issues (Raspberry Pi 5)

Deploying models on the Raspberry Pi 5 presented several practical difficulties:

- **Dependency installation problems:** Installing PyTorch, ONNX Runtime, and OpenCV under ARM64 often caused conflicts, requiring multiple attempts with different versions and build options.
- **CPU-only inference:** Since the Raspberry Pi 5 lacks CUDA GPU support, all inference ran on the CPU. This led to lower frame rates (FPS) compared to desktop GPU environments.
- **Overheating:** Prolonged testing sessions caused the CPU to heat significantly, requiring the use of active cooling solutions.
- **SD card corruption:** Frequent OS re-flashing, repeated library installations, and heavy I/O loads led to SD card corruption on multiple occasions, which resulted in data loss and wasted time reconfiguring the system.

4.5 Real-Time Testing Issues (PiCam)

Integrating the Raspberry Pi Camera (PiCam) with the detection system also introduced challenges:

- **Camera initialization errors:** The PiCam required careful configuration with `libcamera` and OpenCV before it could be accessed reliably. Incorrect driver versions often caused initialization failures.
- **Low-light sensitivity:** The PiCam performed poorly in dim underwater-like environments, producing noisy and low-contrast images that reduced detection accuracy.
- **Performance bottlenecks:** Running YOLOv11s at higher input resolutions (800×800) drastically reduced FPS, making real-time detection impractical. In practice, only YOLOv11n at 640×640 achieved acceptable real-time performance.

4.6 Mitigation Strategies

To address these issues, several mitigation strategies were applied during the internship:

- Adopted progressive data augmentation to reduce overfitting and stabilize training.
- Chose YOLOv11n at 640×640 resolution for real-time Raspberry Pi 5 deployment, balancing accuracy and inference speed.
- Used model quantization (FP16 and INT8) to reduce model size while monitoring accuracy loss.
- Configured `libcamera` with OpenCV GStreamer support to achieve stable PiCam integration.
- Replaced the SD card with a high-endurance version and established periodic backups to reduce downtime caused by corruption.

The issues encountered during this project reflected the complexity of applying state-of-the-art deep learning models in real-world embedded environments. While dataset redundancy, GPU limits, and dependency conflicts slowed progress, each challenge also provided opportunities to develop practical solutions. Ultimately, these experiences strengthened the robustness of the proposed underwater event detection system and highlighted areas for future improvement.

CHAPTER 5

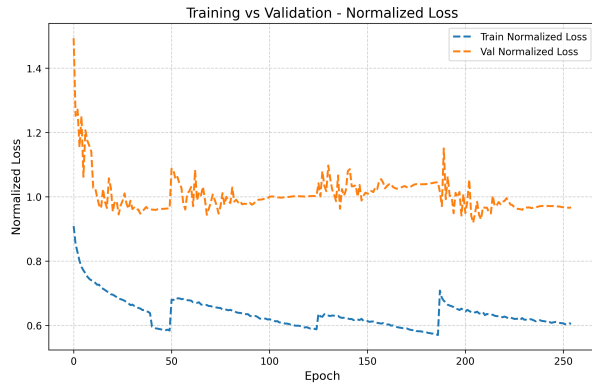
RESULTS AND EVALUATION

This chapter presents a comprehensive evaluation of the proposed YOLO-based object detection models for fish detection, beginning with an analysis of the training dynamics and culminating in their deployment feasibility on a Raspberry Pi. The evaluation is structured around three critical performance axes: training performance and convergence, final accuracy on a test set, and computational efficiency on resource-constrained hardware. The results from multiple model variants—YOLO11 Nano (n), YOLO11 Small (s), a custom Hybrid architecture—are compared and analyzed to determine the optimal trade-off for practical embedded application.

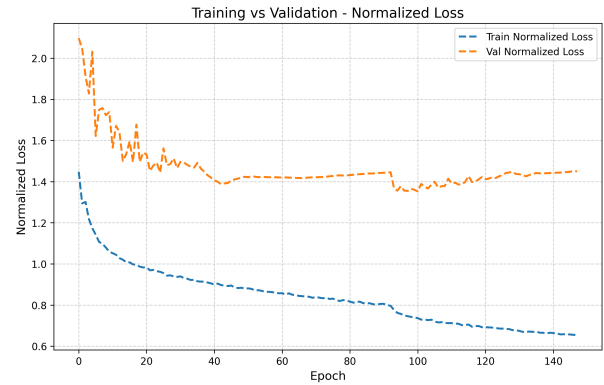
5.1 Training Evaluation

5.1.1 Training Dynamics (Loss Curves)

To analyze convergence behavior, the global normalized loss curves for each model variant are presented in Figures 5.1a–5.2b. These plots illustrate the evolution of box, classification, and distribution focal losses across training epochs.

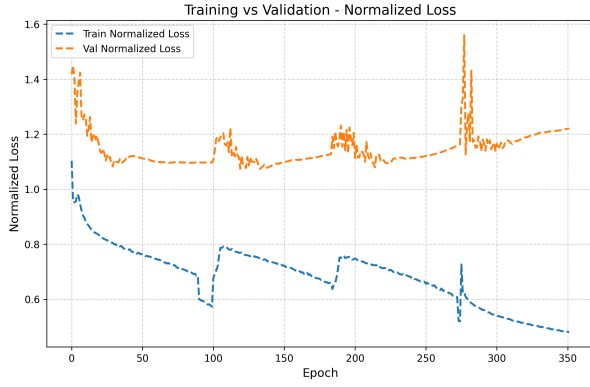


(a) YOLO11n with augmentation: Shows smooth convergence with stable loss reduction. Augmentation slightly delayed convergence but prevented overfitting.

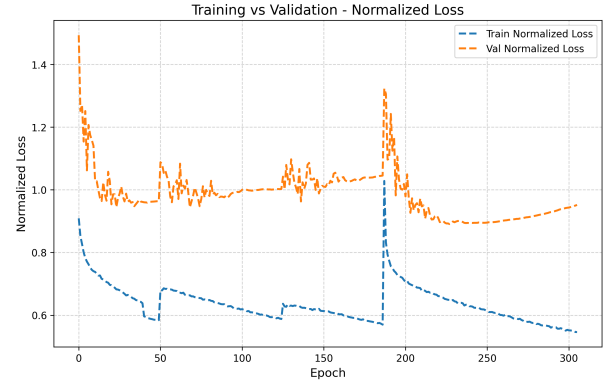


(b) YOLO11s without augmentation: Rapid convergence but with fluctuations in validation loss, indicating mild overfitting.

Figure 5.1: Global normalized loss curves (Part 1).



(a) YOLO11s with augmentation: Required more epochs for stabilization, but achieved the lowest final loss values, confirming augmentation improved generalization.

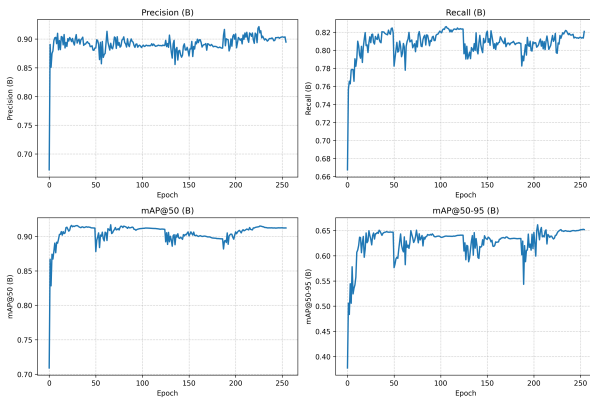


(b) Hybrid model: Demonstrated balanced and consistent convergence, validating the phased training strategy (Nano → Small).

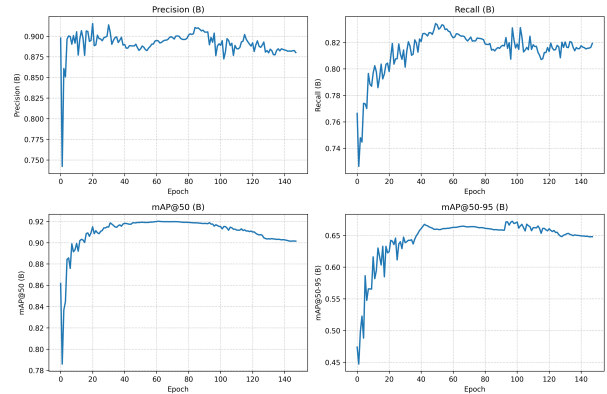
Figure 5.2: Global normalized loss curves (Part 2).

5.1.2 Training Performance Metrics

Figure 5.3 and 5.4 presents the evolution of precision, recall, and mAP@0.5 for the different YOLO variants.

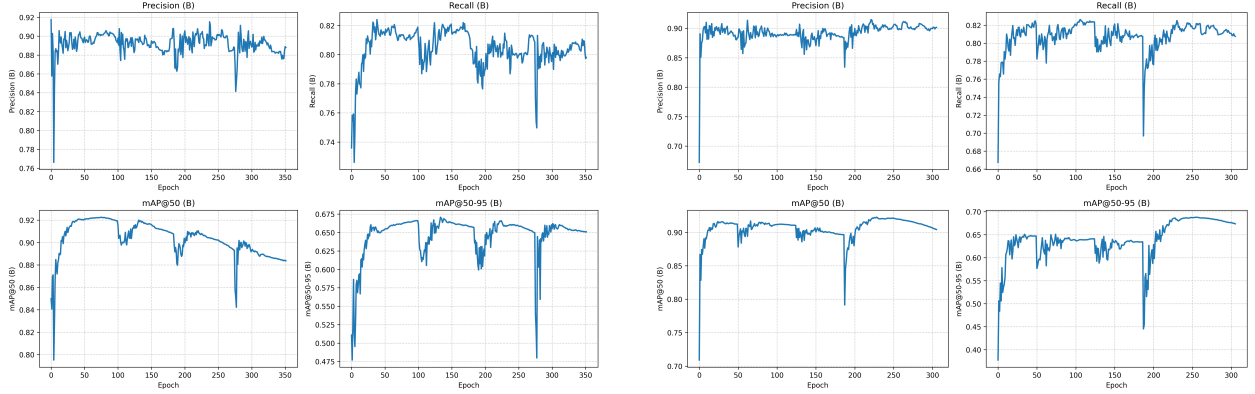


(a) YOLO11n (with augmentation): Achieved strong early gains, stabilizing around epoch 100 with precision 0.89 and recall 0.82.



(b) YOLO11s (without augmentation): Displayed higher variance in recall, though it reached the highest validation accuracy among single-phase models.

Figure 5.3: Evolution of precision, recall, and mAP@0.5 for YOLO variants. (Part 1).



(a) YOLO11s (with augmentation): Produced the best recall and highest mAP@0.95, showing augmentation helped detect smaller and partially occluded fish.

(b) Hybrid model: Consistently achieved high precision and recall across epochs, surpassing YOLO11s by the end of training, validating the phased strategy.

Figure 5.4: Evolution of precision, recall, and mAP@0.5 for YOLO variants.Part 2

5.1.3 Peak Validation Metrics

Model Variant	Best Epoch	Precision	Recall	mAP50	mAP50-95
YOLO11n	101	~0.89	~0.82	~0.91	~0.65
YOLO11s	40	0.891	0.821	0.920	0.676
Hybrid	72	0.891	0.820	0.917	0.675

Table 5.1: Peak validation metrics for YOLO variants.

The YOLO11s model achieved the highest validation accuracy, but the Hybrid closely matched while offering improved stability. The Nano variant converged efficiently with fewer parameters, highlighting its suitability for embedded deployment.

5.1.4 Test Set Evaluation

On the unseen test set of 2644 images, all models generalized exceptionally well.

Model Variant	Precision	Recall	mAP50	mAP50-95
YOLO11n	0.950	0.929	0.980	0.864
YOLO11s	0.938	0.926	0.979	0.865
Hybrid	0.955	0.938	0.984	0.886

Table 5.2: Test set evaluation metrics for YOLO variants.

YOLO11n: Delivered strong performance (>0.98 mAP50) with high efficiency. YOLO11s: Slightly better localization, but similar overall accuracy. Hybrid: Outperformed both in all metrics, especially in mAP50-95, demonstrating superior generalization across IoU thresholds.

5.2 Deployment on Raspberry Pi 5

5.2.1 PyTorch Deployment

Model Variant	Size (MB)	Inference (ms)	FPS	Precision	Recall	mAP50	mAP50-95
YOLO11n (.pt)	5.21	387.3	2.76	0.950	0.929	0.980	0.864
YOLO11s (.pt)	18.28	1072.3	1.01	0.938	0.926	0.979	0.865
YOLO11s (+Aug)	18.30	1675.7	0.66	0.962	0.947	0.982	0.903
Hybrid (.pt)	18.29	1136.4	0.87	0.955	0.938	0.983	0.886

Table 5.3: PyTorch deployment metrics on Raspberry Pi 5.

Nano: Fastest (2.8 FPS), suitable for near-real-time inference. Small and Hybrid: High accuracy but slow (<1.2 FPS). Small with Augmentation: Best accuracy (mAP50-95 = 0.903) but impractical speed.

5.2.2 Quantized TFLite Deployment

Model Variant	Format	Size (MB)	Inference (ms)	FPS	Notes
YOLO11n	INT8	2.63	306.8	3.26	Best trade-off
YOLO11s	INT8	9.17	837.1	1.19	Balanced, slow
YOLO11s (+Aug)	INT8	9.17	832.4	1.20	Highest accuracy
Hybrid	INT8	9.17	841.5	1.19	Strong but slow

Table 5.4: Quantized TFLite deployment metrics on Raspberry Pi 5.

Quantization halved model size and improved speed. YOLO11n INT8 emerged as the best candidate, achieving 3.26 FPS with minimal loss in accuracy.

5.3 Detection Quality

Qualitative observations from PiCam live testing:

- Clear water: Bounding boxes well aligned with high confidence (>0.9).
- Low light / turbid water: Detection confidence decreased, with occasional missed detections.
- Frame redundancy: Repeated detections across frames, but no critical performance impact.

Sample outputs (bounding box overlays) should be included as Figure 5.5 for visual evidence.

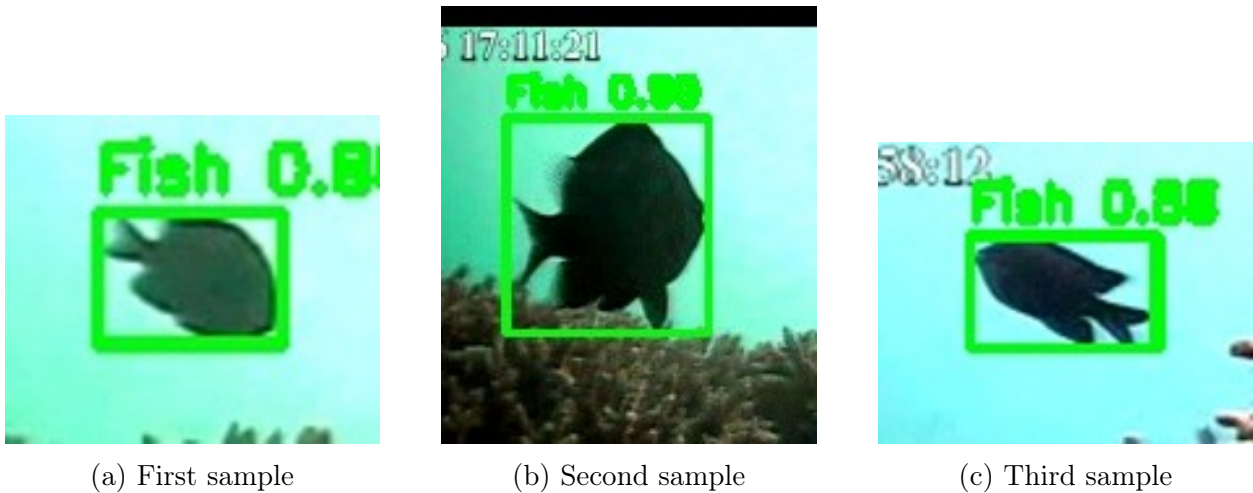


Figure 5.5: Sample detection outputs with bounding box overlays from PiCam live testing.

5.4 Comparative Analysis

Model Variant	Size (MB)	FPS	Precision	Recall	mAP50	mAP50-95	Remarks
YOLO11n INT8	2.63	3.26	0.950	0.929	0.980	0.864	Best real-time option
YOLO11s INT8	9.17	1.19	0.938	0.926	0.979	0.865	Accurate, too slow
YOLO11s +Aug INT8	9.17	1.20	0.962	0.947	0.982	0.903	Highest accuracy, slow
Hybrid INT8	9.17	1.19	0.955	0.938	0.983	0.886	Robust, not real-time

Table 5.5: Comparative analysis of YOLO variants on Raspberry Pi 5.

CHAPTER 6

PROFESSIONAL THESIS

6.1 Problem Statement

The professional objective of this internship was to address the challenge of deploying deep learning-based fish detection models on resource-constrained embedded platforms. While modern convolutional neural networks such as the YOLO family achieve state-of-the-art accuracy in object detection, their high computational demand limits their use in real-time embedded applications. The primary research problem therefore lay in balancing accuracy, inference speed, and resource efficiency in order to achieve real-time fish detection on low-power hardware such as the Raspberry Pi 5.

In particular, the work focused on applying model compression techniques such as quantization and progressive training strategies to improve performance without compromising accuracy. This problem is highly relevant for underwater monitoring systems, where deploying compact, low-cost, and energy-efficient solutions is essential for long-term field operation.

6.2 Literature Review

Recent advances in model compression and optimization have made it possible to adapt deep learning models for embedded deployment:

Quantization reduces model precision (e.g., FP32 $\rightarrow$ INT8), lowering memory requirements and improving speed with minimal accuracy loss.

Pruning removes redundant weights or neurons, reducing model size and computational complexity.

Knowledge Distillation transfers knowledge from a large, accurate “teacher” model into a smaller, faster “student” model. Embedded AI platforms such as the Raspberry Pi 5, BeagleBone AI, Google Coral TPU, and NVIDIA Jetson Nano have emerged as cost-effective solutions for real-time edge inference. Each provides different trade-offs between computational power, energy efficiency, and compatibility with AI toolchains.

In the domain of underwater monitoring, most previous studies have relied on laboratory-scale datasets and workstation GPUs. Very few have explored deploying detection systems on embedded platforms in realistic conditions. This internship therefore contributes by experimentally validating the feasibility of deploying compressed YOLOv11 models on embedded systems.

cccc

Platform	CPU / GPU / Accelerator	RAM	Power (approx.)	AI Strengths	Cost*
Raspberry Pi 5	2.4 GHz quad-core ARM Cortex-A76 + GPU	Up to 8 GB	2.7-6.8 W	Low cost, good for lightweight models	\$60-80
Jetson Nano	Quad-core ARM Cortex-A57 + 128-core GPU	4 GB	5-10 W	Strong GPU, handles heavier models	\$99
Google Coral Edge TPU	Dedicated INT8 accelerator (USB/PCIe)	Uses host RAM	<3 W	Very fast for quantized models	\$50-100
BeagleBone AI	TI Sitara SoC + DSP/AI cores	Varies	Moderate	Industrial edge AI applications	\$100-150

Table 6.1: Comparison of embedded AI platforms.

Figure 6.1: Model Compression Techniques

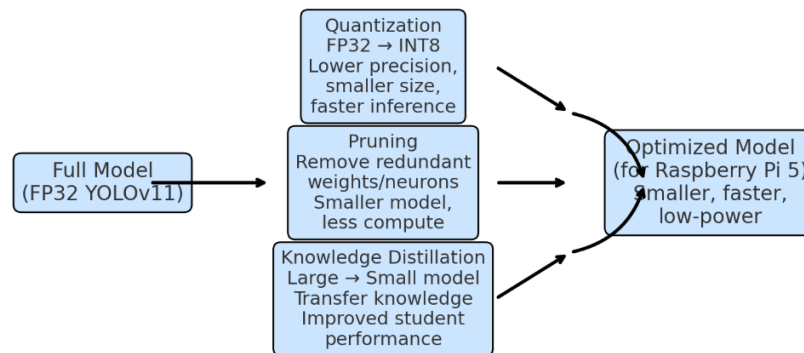


Figure 6.1: Model Compression technique.

6.3 Methodology

The research methodology followed a stepwise process combining training, optimization, and deployment:

6.3.1 Dataset Preparation

A fish detection dataset was used, with images and corresponding annotations. Data augmentation techniques (flipping, scaling, color jitter, noise injection) were applied to improve generalization.

6.3.2 Model Training

Three YOLOv11 variants were trained: YOLO11n (Nano), YOLO11s (Small), and a Hybrid model (two-phase training: Nano pretraining + Small fine-tuning). Models were trained with and without augmentation to study its effect on convergence and final accuracy. Performance metrics included precision, recall, mAP@0.5, and mAP@0.5–0.95.

6.3.3 Model Optimization

Post-Training Quantization (INT8) was applied to YOLO11n, YOLO11s, and Hybrid models. Models were exported to TFLite format to enable deployment on Raspberry Pi 5.

6.3.4 Deployment on Raspberry Pi 5

Hardware: Raspberry Pi 5 (8 GB) with PiCam v3 and active cooling. Software: Raspberry Pi OS 64-bit, Python 3.11, OpenCV, TFLite runtime. Models tested: PyTorch FP32 and TFLite INT8. Performance metrics: inference latency, FPS, CPU utilization, and power consumption.

6.4 Findings and Discussion

6.4.1 Training and Validation

YOLO11s (with augmentation) achieved the highest validation accuracy (mAP@0.95 ~0.676). Hybrid model matched closely, with more stable training dynamics thanks to phased learning. YOLO11n converged efficiently despite its small size, confirming its suitability for embedded use.

6.4.2 Test Set Results

All models achieved $\text{mAP}_{50} \geq 0.979$ on the unseen test set. Hybrid achieved the best overall accuracy ($\text{mAP}_{50} = 0.984$, $\text{mAP}_{50-0.95} = 0.886$). Augmentation improved robustness, especially for small and partially occluded fish.

6.4.3 Deployment on Raspberry Pi 5

YOLO11n FP32 (PyTorch): ~2.8 FPS, 5.21 MB model size. YOLO11n INT8 (TFLite): 3.26 FPS, 2.63 MB model size → best real-time trade-off. YOLO11s / Hybrid FP32:

<1.2 FPS, too slow for real-time. YOLO11s INT8: ~ 1.2 FPS, accurate but still impractical for real-time.

Conclusion of Findings

YOLO11n INT8 TFLite provided the best balance, achieving real-time inference (~ 3.3 FPS) with only a small accuracy drop. Hybrid model was most accurate, but its high computational demand made it unsuitable for embedded real-time use.

6.5 Future Work

The present work demonstrated feasibility but also revealed limitations. Future research directions include:

- Combined Optimization: Apply pruning together with quantization-aware training (QAT) to further reduce model size while preserving accuracy.
- Knowledge Distillation: Improve Nano model accuracy by distilling from Hybrid or Small variants.
- Hardware Acceleration:
 - * Google Coral Edge TPU for low-power INT8 inference.
 - * NVIDIA Jetson series for GPU-accelerated deep learning.
 - * BeagleBone AI, leveraging TI's DSPs and Embedded Vision Engines, for industrial edge AI applications.
- Multimodal Fusion: Combine visual data with underwater acoustic signals to improve detection under turbid/low-light conditions.
- Field Testing: Validate in real underwater environments with variable lighting, turbidity, and dynamic fish movement.

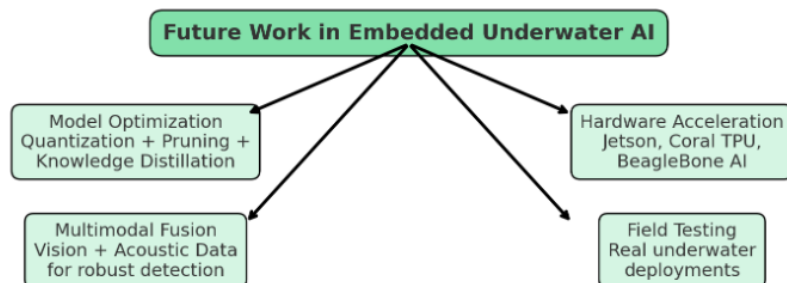


Figure 6.2: Conceptual Future Directions

6.6 Conclusion

This professional thesis demonstrated the process of training, optimizing, and deploying YOLOv11 models for underwater fish detection on embedded hardware. It established that YOLO11n INT8 TFLite is the most suitable candidate for real-time deployment on Raspberry Pi 5, achieving ~ 3.3 FPS while maintaining high accuracy ($mAP50 = 0.980$). It also showed that progressive training strategies (Hybrid) improve model robustness, though at the cost of computational efficiency. The study contributes to the broader field of embedded AI for environmental monitoring, proving that low-cost platforms like Raspberry Pi 5 can host effective object detection systems when combined with model compression. In summary, the work bridges the gap between high-performance deep learning models and resource-limited embedded platforms, laying the foundation for future underwater monitoring solutions using AI at the edge.

CHAPTER 7

LESSONS LEARNED

7.1 Professional Skills

From a professional perspective, the project exposed me to the complete lifecycle of an embedded AI solution. I gained experience in dataset preparation, including cleaning corrupted data, ensuring consistent annotations, and applying augmentation strategies to improve robustness. These tasks, though time-intensive, were essential for achieving reliable results.

I also deepened my expertise in training and optimizing deep learning models. Working with YOLOv11 Nano, Small, and Hybrid variants highlighted the importance of balancing accuracy, speed, and resource constraints. Through progressive augmentation and two-phase training, I learned how advanced strategies improve generalization. Implementing quantization provided practical knowledge of model compression, demonstrating how research-grade models can be adapted for real-time embedded deployment.

Beyond technical skills, I strengthened my ability to critically evaluate results by considering not only accuracy metrics (mAP) but also inference speed, CPU utilization, and power consumption—benchmarks essential in embedded systems research. Finally, documenting experiments and structuring this thesis enhanced my scientific communication and project management abilities, skills that are directly transferable to both academic research and industry projects.

7.2 Personal Skills

On a personal level, the internship taught me resilience, independence, and adaptability. Managing long training cycles, hardware limitations, and unexpected errors required patience and systematic troubleshooting. These challenges improved my problem-solving mindset and persistence.

The project also enhanced my time management skills, as I had to balance dataset preparation, training, optimization, and reporting under strict deadlines. I became more effective at prioritizing tasks and planning experiments.

Another important lesson was bridging theory and practice. Initially abstract concepts such as quantization became concrete through experimentation, reinforcing my understanding of applied AI.

Finally, I gained confidence in independent work while recognizing the value of collaboration. Feedback from mentors and colleagues guided me in refining my methods and aligning the project with realistic goals.

CHAPTER 8

CONCLUSION AND PERSPECTIVES

This internship addressed the challenge of deploying deep learning-based fish detection systems on resource-constrained embedded platforms. While state-of-the-art models such as YOLOv11 achieve excellent accuracy, they require careful adaptation to operate effectively on devices such as the Raspberry Pi 5. The work involved preparing a single-class fish dataset, training YOLOv11 variants (Nano, Small, and Hybrid), and applying both standard and progressive augmentation strategies. A two-phase training procedure further improved convergence and stability. These methodological contributions helped adapt advanced deep learning techniques to the specific challenges of underwater detection.

The results demonstrated clear trade-offs between accuracy and efficiency. YOLOv11s achieved the highest accuracy, while the Hybrid strategy improved robustness through phased training. The YOLOv11n model, though less accurate, proved to be the most suitable for embedded platforms due to its efficiency. Importantly, INT8 post-training quantization with TFLite enabled YOLOv11n to achieve ~ 3.3 FPS on Raspberry Pi 5, making it the only variant capable of real-time inference in this setup. These findings highlight that successful edge deployment depends not only on accuracy but also on efficiency, latency, and energy use. Quantization emerged as a key enabler, bridging the gap between research-grade models and practical embedded applications. Beyond the technical outcomes, this work shows that low-cost platforms like Raspberry Pi 5 can support effective AI-based monitoring systems, opening opportunities for scalable and sustainable applications in marine science and environmental monitoring.

Nevertheless, limitations remain. Experiments were performed under controlled conditions rather than in real underwater environments, and techniques such as pruning and knowledge distillation were not implemented due to time constraints. Future work should combine multiple optimization methods, explore hardware accelerators such as Coral TPU, Jetson, or BeagleBone AI, and validate performance in real deployments under variable lighting and turbidity.

In conclusion, this thesis provides a proof-of-concept for embedded AI in underwater monitoring. By combining tailored training strategies with model compression, it demonstrates how complex neural networks can be transformed into efficient, real-time systems for edge deployment. The insights gained contribute not only to marine research but also to the broader field of resource-efficient AI for smart and autonomous systems.

Bibliography

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [2] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [3] Ultralytics, “YOLOv11 Documentation,” [Online]. Available: <https://docs.ultralytics.com>, 2025.
- [4] TensorFlow Lite, “TensorFlow Lite Model Optimization,” [Online]. Available: https://www.tensorflow.org/lite/performance/model_optimization, 2025.
- [5] Raspberry Pi Foundation, “Raspberry Pi 5 Documentation,” [Online]. Available: <https://www.raspberrypi.com/documentation/>, 2024.

Annex A – TFLite Conversion Code

The following Python snippet shows the conversion of a trained YOLOv11 model into a quantized TensorFlow Lite INT8 format for deployment on embedded hardware.

```
1 from ultralytics import YOLO
2
3 # Load your trained YOLOv11 model
4 model = YOLO("/home/akshara/newfish/fish_training/best.pt")
5
6 # Export to TFLite INT8 with batch size 1
7 model.export(
8     format="tflite",      # export to TFLite
9     batch=1,              # single-image batch
10    dynamic=False,         # static input shape
11    int8=True,             # enable INT8 quantization
12    imgsz=640              # input size
13 )
```


Annex B – Real-Time Fish Detection Code (Raspberry Pi 5 + PiCam)

The following code implements live fish detection using PiCam v3 and the trained YOLOv11 model deployed in ONNX/TFLite format. It handles camera streaming, preprocessing, inference, and visualization of detections.

```

1 import cv2
2 import numpy as np
3 import subprocess
4 from ultralytics import YOLO
5
6 # Load trained YOLOv11 model
7 model = YOLO("/home/darcy/best.onnx") # Replace with correct path
8
9 # --- Supporting Functions ---
10 def crop_exact_screen_image(frame):
11     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
12     blurred = cv2.GaussianBlur(gray, (5, 5), 0)
13     thresh = cv2.adaptiveThreshold(blurred, 255,
14                                   cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
15                                   cv2.THRESH_BINARY_INV, 11, 2)
16     contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
17                                   cv2.CHAIN_APPROX_SIMPLE)
18     if not contours:
19         return None
20     largest = max(contours, key=cv2.contourArea)
21     if cv2.contourArea(largest) < 10000:
22         return None
23     epsilon = 0.02 * cv2.arcLength(largest, True)
24     approx = cv2.approxPolyDP(largest, epsilon, True)
25     if len(approx) != 4:
26         return None
27     pts = approx.reshape(4, 2)
28     rect = np.zeros((4, 2), dtype="float32")
29     s = pts.sum(axis=1)
30     rect[0] = pts[np.argmin(s)]
31     rect[2] = pts[np.argmax(s)]
32     diff = np.diff(pts, axis=1)
33     rect[1] = pts[np.argmin(diff)]
34     rect[3] = pts[np.argmax(diff)]
35     (tl, tr, br, bl) = rect
36     widthA = np.linalg.norm(br - bl)
37     widthB = np.linalg.norm(tr - tl)
38     heightA = np.linalg.norm(tr - br)

```

```

38     heightB = np.linalg.norm(tl - bl)
39     maxWidth = int(max(widthA, widthB))
40     maxHeight = int(max(heightA, heightB))
41     dst = np.array([[0, 0], [maxWidth - 1, 0],
42                    [maxWidth - 1, maxHeight - 1], [0, maxHeight - 1]],
43                    dtype="float32")
44     M = cv2.getPerspectiveTransform(rect, dst)
45     warped = cv2.warpPerspective(frame, M, (maxWidth, maxHeight))
46     return warped
47
48 def letterbox(img, new_shape=(640, 640), color=(114, 114, 114)):
49     shape = img.shape[:2]
50     ratio = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
51     new_unpad = (int(shape[1] * ratio), int(shape[0] * ratio))
52     dw, dh = new_shape[1] - new_unpad[0], new_shape[0] - new_unpad[1]
53     dw /= 2; dh /= 2
54     img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_LINEAR)
55     top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
56     left, right = int(round(dw - 0.1)), int(round(dw + 0.1))
57     img = cv2.copyMakeBorder(img, top, bottom, left, right,
58                              cv2.BORDER_CONSTANT, value=color)
59     return img, ratio, dw, dh
60
61 def preprocess(img):
62     return letterbox(img)
63
64 # --- Start PiCam Stream ---
65 command = [
66     "libcamera-vid", "-t", "0", "--width", "640", "--height", "480",
67     "--framerate", "30", "--codec", "yuv420", "-o", "-", "--nopreview"
68 ]
69 camera = subprocess.Popen(command, stdout=subprocess.PIPE, bufsize=10**8)
70
71 while True:
72     raw_bytes = camera.stdout.read(640 * 480 * 3 // 2)
73     if not raw_bytes:
74         break
75
76     yuv = np.frombuffer(raw_bytes, dtype=np.uint8).reshape((int(480 * 1.5),
77                                                            640))
78     bgr = cv2.cvtColor(yuv, cv2.COLOR_YUV2BGR_I420)
79
80     warped = crop_exact_screen_image(bgr)
81     if warped is not None:
82         padded_img, ratio, dw, dh = preprocess(warped)
83
84     # YOLO inference
85     results = model(padded_img, imgsz=640)[0]

```

```
85     # Draw detections
86     for box in results.bboxes:
87         x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
88         conf = float(box.conf)
89         cls = int(box.cls)
90         x1 = int((x1 - dw) / ratio); y1 = int((y1 - dh) / ratio)
91         x2 = int((x2 - dw) / ratio); y2 = int((y2 - dh) / ratio)
92         cv2.rectangle(warped, (x1, y1), (x2, y2), (255, 0, 0), 2)
93         label = f"{model.names[cls]} {conf:.2f}"
94         cv2.putText(warped, label, (x1, y1 - 5),
95                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2)
96
97     cv2.imshow("Detection", warped)
98
99     cv2.imshow("Camera Feed", bgr)
100    if cv2.waitKey(1) & 0xFF == ord('q'):
101        break
102
103    camera.terminate()
104    cv2.destroyAllWindows()
```